

Building My Honeypot

(Windows VM + MySQL + Microsoft Defender for Endpoint + sentinel)

Overview

For this project I built a honeypot on Azure on purpose: a Windows 11 VM running MySQL that I would eventually weaken and open up to the whole internet, so real attackers would find it and try to break in. The twist is the order I did things in. I built and locked it down first, hooked up logging, captured a clean baseline, wrote my detection rules while everything was still quiet, and only after that did I make it vulnerable and expose it. That way, when something happened, my own alerts would catch it.

This report only covers the setup, everything up to the moment the honeypot went live. What the attackers actually did and how I investigated it is in my separate incident report.

Here is the overall picture of what I built:

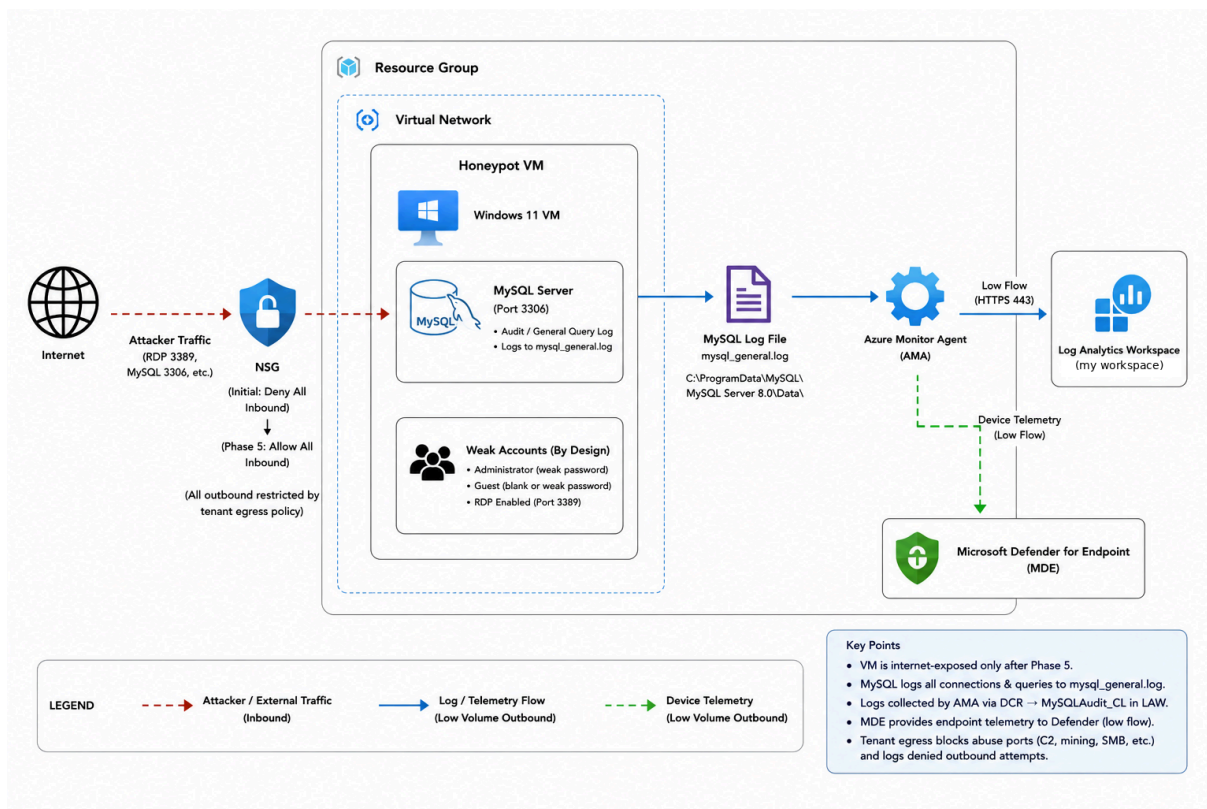


Figure 1: Honeypot architecture

In short: attacker traffic comes in through the NSG (which starts as deny-all and gets opened later) to the Windows 11 VM. MySQL writes every connection and query to a log file, the Azure Monitor Agent picks that file up and sends it to my Log Analytics workspace, and Microsoft Defender for Endpoint sends the device telemetry to the same place. Outbound traffic from the VM is heavily restricted by the tenant, so if the box does get breached, things like C2, mining or pivoting attempts get blocked and logged instead of actually working.

What I used

- Microsoft Azure (VM, NSG, resource group, virtual network)
- Windows 11 VM, MySQL Server 8.0 and MySQL Workbench
- Microsoft Defender for Endpoint (MDE)
- Azure Monitor Agent (AMA) with a Data Collection Rule
- Log Analytics workspace and Microsoft Sentinel
- KQL for all the queries and detection rules

Phase 1: Build the VM (locked down first)

I wanted the machine fully locked down while I was building it, so nothing from the internet could touch it during setup.

1. Deployed a **Windows 11 VM** in my own resource group, with a strong username and password, and made sure it had a **public IP address**.
2. Named the VM **corp-ae1-872d**. I picked something that looks like a normal company machine instead of something obvious like "test-vm-1", so it looks more believable to anyone scanning.
3. Set the NSG to **deny all inbound traffic** from the internet.
4. Onboarded the VM to **Microsoft Defender for Endpoint** and checked that it showed up in the **DeviceInfo** table in my workspace.

A quick check to make sure the VM was reporting in (note that MDE cuts long device names, so I used a short match):

```
DeviceInfo
| where DeviceName startswith "corp-ae1"
| project TimeGenerated, DeviceName, OSPlatform, PublicIP
```

Phase 2: Install and populate MySQL

Installing MySQL

1. Installed the **Microsoft Visual C++ 2019 Redistributable (x64)** first, since MySQL needs it.
2. Installed **MySQL Server 8.0** using the **Developer Default** option so I would also get MySQL Workbench. Everything else was left on default.
3. For the MySQL root password I used a strong one at this stage and saved it in my password manager. (I change this later in Phase 5.)
4. Opened **MySQL Workbench**, created a new connection and connected to the server.

Adding dummy data

5. Downloaded the **db_info_import.sql** script onto the VM, opened it in Workbench and ran it. This creates the **lnp_corp** database with fake customer data, so the attackers have something that looks worth stealing.

6. Workbench looks frozen while it imports. It is not, it just takes a while. If the connection drops or it fails, enter the password again and rerun it. Lowering the number of users in the script from 5000 to 1000 helps if it keeps failing.
7. Refreshed the **Schemas** tab and confirmed the new **Inp_corp** schema was there.

Turning on MySQL logging

This is the most important part of the build, because everything I detect later depends on these logs. I enabled the general query log so every connection (successful and failed) and every query gets written to a file:

```
SET GLOBAL general_log = 'ON';  
SET GLOBAL log_output = 'FILE';  
SHOW VARIABLES LIKE 'general_log%';
```

Then I replaced the **my.ini** file at C:\ProgramData\MySQL\MySQL Server 8.0\my.ini with a version that does two things: it logs everything to the file below, and it allows MySQL logins over the network.

```
C:\ProgramData\MySQL\MySQL Server 8.0\Data\mysql_general.log
```

After that I restarted the **MySQL80** service from services.msc, ran a few SELECT queries in Workbench and opened **mysql_general.log** to make sure my queries were showing up. They were, so I noted the log path because I need it for the next phase.

As an extra, I also made a full backup of all the databases on the server, so I could restore the original data if I ever needed to.

Phase 3: Send the logs to Log Analytics

Now I needed the MySQL log file to land in my workspace so Sentinel can query it. First I made sure the VM was **on and running** (this matters, the agent does nothing if the VM is off) and confirmed that device telemetry was still flowing into the DeviceInfo table.

Then I created a **Data Collection Rule (DCR)** with a **custom text log** data source pointing the Azure Monitor Agent at the MySQL log file. These are the exact settings I used:

- **Data source type:** Custom Text Logs
- **File pattern:** C:\ProgramData\MySQL\MySQL Server 8.0\Data\mysql_general.log
- **Table name:** MySQLAudit_CL
- **Record delimiter:** TimeStamp
- **Timestamp format:** ISO 8601

Figure 2: Data Collection Rule data source settings

For the destination I picked my Log Analytics workspace. These settings have to be exactly right or nothing shows up, and it can take up to 30 minutes for the first logs to appear, so I was patient here.

Right after the DCR was created, the **Azure Monitor Agent** got installed on the VM automatically. I checked this under the VM, **Settings > Extensions + applications**, and the **AzureMonitorWindowsAgent** extension was there with "Provisioning succeeded".

To verify ingestion I queried the new table. The table holds logs from every machine sending to the workspace, so I filtered to my own VM using the resource ID:

```
MySQLAudit_CL
| project TimeGenerated, RawData, _ResourceId
| where _ResourceId endswith "corp-ae1-872d"
```

My test connections and SELECT queries showed up, which confirmed the whole pipeline works: MySQL, then the log file, then AMA, then the workspace. I also checked that the core device tables (DeviceLogonEvents and the others) were populating before moving on.

Phase 4: Write the detections (while the box is still clean)

This was the part I was most careful about. I wrote the detection rules **before** exposing anything, and I confirmed that they stayed quiet against the clean baseline. If I had waited until after the attack, I would be building alerts around something I had already missed. I made two analytics rules in Microsoft Sentinel.

Rule 1: Successful logon to the VM

This rule fires when someone successfully logs on to the VM as **administrator** or **guest** from a remote IP. It returns nothing right now, because the administrator account does not exist yet in my VM and the guest account is still disabled. Both get set up in Phase 5, and from then on any successful logon will make this rule trigger.

```
// Virtual Machine Logons
let MyDevice = "corp-ae1-872d"; // MDE truncates/cuts off long device names
DeviceLogonEvents
| where DeviceName == MyDevice
| where AccountName in~ ("administrator", "guest")
| where ActionType == "LogonSuccess"
| project TimeGenerated, RemoteIP, AccountName, DeviceName, ActionType, LogonType
```

In the rule settings I mapped two entities so the incidents are easy to read: **Host** (HostName = DeviceName) and **IP** (Address = RemoteIP). The rule runs every **10 minutes** and looks back at the last 10 minutes, and it starts running automatically.

Alert enhancement

Entity mapping

Map up to 10 entities recognized by Microsoft Sentinel from the appropriate fields available in your query results. This enables Microsoft Sentinel to recognize and classify the data in these fields for further analysis. For each entity, you can define up to 3 identifiers, which are attributes of the entity that help identify the entity as unique. [Learn more >](#)

Host
HostName DeviceName + Add identifier

IP
Address RemoteIP + Add identifier

+ Add new entity

> Custom details

> Alert details

Query scheduling

Run query every *
10 Minutes

Lookup data from the last *
10 Minutes

Start running ⓘ

☒ Automatically

☐ At specific time (Preview)

6/23/2026 12:00 PM

Figure 3: Entity mapping and scheduling for the VM logon rule

Rule 2: Successful login to MySQL

The MySQL log lines come in as one big **RawData** string, so before I could use them I had to parse them into proper columns (username, IP and so on). The query also needs to separate failed logins from successful ones: it first collects the connection IDs that had an "Access denied", then labels everything else as a success.

```
// MySQL - Failed Login -> Successful Login Detection
let MyDevice = "corp-ae1-872d";
```

```

let MyTimeframe = ago(7d);
let FailedConnections =
MySQLAudit_CL
| where TimeGenerated > MyTimeframe
| extend RawData = replace_string(RawData, "\t", " ")
| extend DeviceName = tostring(split(_ResourceId, "/")[-1])
| where DeviceName == MyDevice
| where RawData has "Access denied"
| extend ConnectionId = extract(@"^S\s+(\d+)\s+Connect", 1, RawData)
| distinct ConnectionId;
MySQLAudit_CL
| where TimeGenerated > MyTimeframe
| extend RawData = replace_string(RawData, "\t", " ")
| extend DeviceName = tostring(split(_ResourceId, "/")[-1])
| where DeviceName == MyDevice
| where RawData has "Connect"
| extend ConnectionId = extract(@"^S\s+(\d+)\s+Connect", 1, RawData)
| extend ActionType =
    case(
        RawData has "Access denied", "LogonFailure",
        ConnectionId in (FailedConnections), "Ignore",
        "LogonSuccess")
| where ActionType != "Ignore"
| extend Username = replace_string(tostring(split(tostring(split(RawData, "@")[0]), "
")[-1]), "'", "")
| extend IpAddress = replace_string(tostring(split(split(RawData, "@")[1], " ")[0]),
"',')
| where ActionType == "LogonSuccess"
| project TimeGenerated, DeviceName, Username, IpAddress, ActionType, RawData
| order by TimeGenerated desc

```

Entity mapping for this one: **IP** (Address = RemoteHost) and **Host** (HostName = DeviceName), with the same schedule as the first rule, every 10 minutes with a 10 minute lookback.

Alert enhancement

Entity mapping

Map up to 10 entities recognized by Microsoft Sentinel from the appropriate fields available in your query results. This enables Microsoft Sentinel to recognize and classify the data in these fields for further analysis. For each entity, you can define up to 3 identifiers, which are attributes of the entity that help identify the entity as unique. [Learn more >](#)

IP

Address RemoteHost + Add identifier

Host

HostName DeviceName + Add identifier

+ Add new entity

Custom details

Alert details

Query scheduling

Run query every *

10 Minutes

Lookup data from the last *

10 Minutes

Start running ⓘ

☒ Automatically

☐ At specific time (Preview)

6/23/2026 12:00 PM

< Previous Next : Incident settings >

Figure 4: Entity mapping and scheduling for the MySQL logon rule

Both rules were saved, enabled and quiet against the clean baseline. That meant that the first real alert would be a real event and not noise.

Phase 5: Weaken the box and expose it

Only after both rules were armed did I start making the VM vulnerable. I did this in a specific order, so I would know exactly what was open at each step.

Weakening the Windows side

1. Using **compmgmt.msc**, I enabled the built-in **Administrator** account, made sure it was in the **Administrators** group, and gave it a very weak password, the kind that sits at the top of common password lists.
2. Enabled the **Guest** account in **compmgmt.msc**, set its password to **blank**, and added it to the **Users** group.
3. Allowed Guest to log on over the network. In **secpol.msc > Local Policies > User Rights Assignment**:
 - **Deny log on through Remote Desktop Services**: removed Guest from this list (it is there by default).

- **Allow log on through Remote Desktop Services:** made sure Guest (or Remote Desktop Users) is on the list.
 - **Deny log on locally:** checked it and removed Guest if it was there.
 - Under **Security Options**, set **Accounts: Limit local account use of blank passwords to console logon only** to **Disabled**.
4. In the Remote Desktop settings, added the Guest account to **Remote Desktop Users**, then ran **gpupdate /force** to apply everything.

Weakening MySQL

I made MySQL reachable from the network with a trivially guessable root login. This creates a separate root account that can log in from anywhere, with the password "root":

```
CREATE USER 'root'@'%' IDENTIFIED BY 'root';  
GRANT ALL PRIVILEGES ON *.* TO 'root'@'%' WITH GRANT OPTION;  
FLUSH PRIVILEGES;
```

Note that the original local root account (with the strong password from Phase 2) still exists too. This new one is a separate account that can authenticate over the network.

Opening it up

5. Before touching the network, I captured an **Investigation Package** for the VM from Microsoft Defender. This is a snapshot of the machine in its clean state that I can compare against later.
6. **Disabled the Windows Firewall** on the VM.
7. **Changed the NSG to allow all inbound traffic.** Both RDP (3389) and MySQL (3306) are now reachable from anywhere, which makes the box much easier to find.
8. Recorded the exact exposure time, which marks the start of the incident window: **[add exposure timestamp here]**
9. Double checked that both Sentinel analytics rules were enabled and armed before walking away.

Running it

The VM is set to shut down automatically every night at midnight Eastern time (this is for cost saving, and it also acts as a soft containment). It does not turn itself back on, so I started it again each morning.

One design note: the main path I expected was **RDP first, then pivot to MySQL**. Attackers brute force RDP, get in, then find the database on the same machine and go after the data. That gives a better story (initial access, discovery, collection) than someone just hitting port 3306. I left 3306 open as well as insurance, so the database would get touched during my observation window either way.

Where things stand

At this point the honeypot was live: a believable Windows 11 machine with weak accounts and an open MySQL database, full logging into my workspace, two armed detection rules, and a clean baseline to compare against. What happened after that, including the breach, the investigation and my response, is covered in the separate incident report.